

# Federated Edge-Cloud Resilience Architecture for Distributed Campus Networks Using Quantized TCN-GRU Sequence Detectors and Deep Reinforcement Learning for SDN Control

Trinh Hoang Tu\* and Cao Tien Thanh†

*Faculty of Information Technology*

*Ho Chi Minh City University of Foreign Languages - Information Technology*

*Ho Chi Minh City, Vietnam*

Email: \*tth.csec2005@gmail.com, †thanht@hufit.edu.vn

**Abstract**—Modern multi-campus higher education infrastructures face escalating cyber threats driven by high-density IoT deployments, unmanaged student endpoints, and massive distributed telemetry flows. Centralized security architectures suffer from wide-area bandwidth saturation, student data privacy vulnerabilities, and extended containment latency. In this paper, we propose a privacy-aware, event-driven Federated Edge-Cloud Resilience Architecture engineered specifically for distributed campus environments. The platform deploys lightweight PyTorch dynamically INT8-quantized Temporal Convolutional Network and Gated Recurrent Unit (TCN-GRU) sequence detectors directly at edge gateways, executing localized anomaly scoring and participating in collaborative Federated Averaging (FedAvg) aggregation without raw telemetry egress. Enriched high-confidence security events are published over an asynchronous Redis Streams bus to a cloud-based dynamic risk engine and a Deep Q-Network (DQN) Software-Defined Networking (SDN) controller for automated closed-loop containment. Evaluated across  $K = 5$  federated campus clients on the InSDN benchmark with additional cross-dataset zero-shot evaluation on CSE-CIC-IDS2018, the proposed framework achieves a binary anomaly detection F1-score of 0.8582 (ROC-AUC = 0.9729, PR-AUC = 0.9567) under uniform partitioning, outperforming isolated uncoordinated local baselines (0.8314). Under non-IID Dirichlet-skewed ( $\alpha = 0.5$ ) client attack distributions, the global model achieves  $F1 = 0.8097$ , demonstrating stable collaborative convergence despite statistical heterogeneity. The edge compute pipeline completes in  $8.290 \pm 0.140$  ms (with single-sequence INT8 scoring in 4.555 ms), and automated closed-loop OpenFlow mitigation completes in  $12.34 \pm 0.55$  ms on Open vSwitch datapaths. Evaluated over 100 test episodes across 5 random seeds (500 total evaluation episodes), the learned DQN policy achieves a  $98.4 \pm 0.6\%$  threat containment rate with only  $2.1 \pm 0.4\%$  false quarantine, substantially outperforming static threshold heuristics ( $14.8 \pm 1.2\%$  false isolation). Furthermore, dynamic INT8 quantization compresses the GRU and linear model footprint to 0.027 MB (27.5 KB), maintaining minimal edge CPU utilization (13.8%) under sustained throughput of 10,000 events/sec.

**Index Terms**—Federated Learning, Edge Computing, INT8 Quantization, TCN-GRU Sequence Detector, Software-Defined Networking, Deep Reinforcement Learning, Campus Network Security.

## I. INTRODUCTION

Higher education institutions operate geographically distributed campus networks connecting academic departments, student residential dormitories, open research laboratories, and centralized administrative databases. The proliferation of unmanaged Bring-Your-Own-Device (BYOD) endpoints and high-density Internet of Things (IoT) sensors (e.g., smart surveillance, building automation, environmental monitoring) significantly expands the perimeter attack surface. Traditional enterprise defense paradigms rely on centralized Security Information and Event Management (SIEM) platforms and inline Deep Packet Inspection (DPI) proxies. However, this centralized approach exhibits critical architectural limitations in modern multi-campus environments:

- 1) **Privacy Concerns and WAN Bandwidth Saturation:** Continuous backhauling of raw packet traces or granular NetFlow streams across Wide Area Network (WAN) links saturates satellite inter-campus uplinks and risks violating regional student data privacy mandates.
- 2) **High Response Latency and Alert Fatigue:** Centralized manual triage workflows in Security Operations Centers (SOC) can experience substantial response delays and triage bottlenecks during aggressive volumetric or lateral threat propagation (e.g., ransomware, ARP poisoning, credential brute-forcing).

To address these challenges, we present a system-level co-design of **Federated Edge-Cloud Resilience Architecture** tailored for distributed campus networks. Our system decouples threat detection and mitigation into a coordinated 4-tier event-driven hierarchy: (i) localized edge telemetry extraction with PyTorch dynamically INT8-quantized sequence detectors; (ii) privacy-aware Federated Averaging (FedAvg) model parameter coordination; (iii) asynchronous Redis Streams event routing coupled with spatial-temporal identity-context fusion; and (iv) closed-loop Deep Q-Network (DQN) Software-Defined Networking (SDN) policy enforcement.

The key contributions of this paper are summarized as follows:

- **Quantized Edge Sequence Detection:** We design a hybrid Temporal Convolutional Network (TCN) and Gated Recurrent Unit (GRU) detector with dynamic INT8 quantization on recurrent and linear layers (0.027 MB footprint), executing sub-5 ms unbatched flow scoring at distributed campus edge taps.
- **Privacy-Aware Collaborative Training:** We implement a FedAvg aggregation protocol across distributed campus clients, demonstrating collaborative training stability under uniform client partitioning ( $F1 = 0.8582$  vs. 0.8314 uncoordinated local baseline) and characterized non-IID Dirichlet evaluation distributions ( $F1 = 0.8097$ ).
- **Context-Enriched Event Pipeline:** We construct an asynchronous event pipeline over Redis Streams incorporating dynamic identity-to-role risk scoring to eliminate transient false alarms.
- **Learned Closed-Loop DQN SDN Containment:** We formulate an adaptive Deep Q-Network (DQN) SDN controller with discrete action spaces (OpenFlow rate-limiting, VLAN quarantine, flow drop) achieving an automated closed-loop mitigation latency of  $12.34 \pm 0.55$  ms with minimal benign user disruption.
- **Empirical System Benchmark:** We present detailed experimental evaluations on InSDN and CSE-CIC-IDS2018 benchmarks, validating per-stage latency, FedAvg convergence, DQN policy trade-offs, edge CPU/RAM scaling up to 10,000 events/sec, and an artifact package prepared for release upon acceptance.

## II. RELATED WORK & NOVELTY POSITIONING

### A. Federated Learning in Network Intrusion Detection

Federated Learning (FL), pioneered by McMahan et al. [1], enables collaborative model training across distributed clients without sharing raw private data. In network security, collaborative FL has been applied to IoT anomaly detection and DDoS mitigation [2], [3]. Kianpisheh and Taleb [3] investigated collaborative FL paired with DRL-based control for 6G DDoS mitigation. However, several representative approaches evaluate unquantized 32-bit floating-point (FP32) recurrent models, which impose prohibitive memory and compute footprints on constrained campus edge gateways.

### B. Sequence Modeling and Edge Model Quantization

Temporal Convolutional Networks (TCN) [4] leverage dilated causal convolutions to capture extended historical receptive fields with lower computational complexity than standard Recurrent Neural Networks (RNN). Recent studies have explored hybrid convolutional and recurrent architectures (e.g., CNN/TCN and GRU) for sequential telemetry modeling in SDN environments [5]. Nevertheless, deploying deep sequential detectors at edge gateways requires post-training dynamic quantization to minimize runtime latency and memory saturation. Our work customizes dynamic INT8 post-training

quantization [13] for TCN-GRU flow detectors, compressing storage to 0.027 MB while preserving detection fidelity.

### C. Automated SDN Mitigation and DRL Control

Software-Defined Networking (SDN) provides programmable centralized control of network forwarding planes via OpenFlow protocols [8], [17]. Integrating Deep Reinforcement Learning (DRL) into SDN controllers enables autonomous selection of mitigation policies balancing security containment against network quality-of-service (QoS) degradation [7], [9]. Yungaicela-Naula et al. [7], [9] demonstrated the feasibility of DRL for autonomous DDoS defense in SDN/NFV environments. Table I positions our framework against representative related architectures.

TABLE I  
COMPARISON WITH REPRESENTATIVE RELATED ARCHITECTURES

| Framework             | Detection Model           | Federated    | Edge INT8      | Event Bus     | Closed-Loop SDN |
|-----------------------|---------------------------|--------------|----------------|---------------|-----------------|
| DeepLog [6]           | LSTM (Next-Event)         | No           | No             | No            | No              |
| Kianpisheh et al. [3] | FP32 GRU                  | Yes          | No             | No            | Partial         |
| Yungaicela et al. [7] | DRL Agent Only            | No           | No             | ZeroMQ        | Yes (SDN)       |
| Ours                  | TCN-GRU (INT8 GRU/Linear) | Yes (FedAvg) | Yes (0.027 MB) | Redis Streams | Yes (DQN-SDN)   |

## III. THREAT MODEL & CAMPUS ATTACK VECTORS

Multi-campus educational networks accommodate diverse user groups (faculty, students, visitors) operating across academic, dormitory, and administrative VLANs. We assume external adversaries and compromised internal BYOD endpoints can launch attacks spanning the STRIDE threat categories [10]. Table II maps targeted campus assets to MITRE ATT&CK techniques and automated mitigation strategies.

TABLE II  
CAMPUS ATTACK VECTORS AND AUTOMATED MITIGATION MAPPING

| Threat (STRIDE)   | Targeted Asset        | MITRE ID | Automated Containment Strategy                         |
|-------------------|-----------------------|----------|--------------------------------------------------------|
| Spoofing          | Rogue Access Point    | T1557    | Identity Fusion MAC/IP state validation                |
| Tampering         | IoT Gateway Telemetry | T1565    | Edge TCN-GRU sequence anomaly detection                |
| Repudiation       | Authentication Portal | T1078    | Dynamic Risk Resolver escalation & audit logging       |
| Info Disclosure   | Academic Research DB  | T1048    | Role-aware flow bandwidth throttling ( $a_1$ )         |
| Denial of Service | Core Campus Switch    | T1498    | Automated OpenFlow meter drop / BGP Flowspec ( $a_3$ ) |
| Elevation of Priv | Domain Controller     | T1068    | 802.1X quarantine VLAN port reassignment ( $a_2$ )     |

## IV. FEDERATED EDGE-CLOUD SYSTEM ARCHITECTURE

### A. Architectural Overview

As illustrated in Fig. 1, the platform operates across four coordinated tiers: (i) **Tier 1 (Edge Gateways):** Distributed campus taps vectorize ingress flow records and execute local INT8 TCN-GRU sequence scoring; (ii) **Tier 2 (Event Bus & FL Coordinator):** High-throughput Redis Streams buffer security events while the FedAvg coordinator aggregates client model weights; (iii) **Tier 3 (Cloud Risk Engine & DRL Control):** Enriched telemetry is evaluated by an identity-context risk resolver and Deep Q-Network (DQN) policy agent; (iv) **Tier 4 (SDN Actuation):** Programmatic OpenFlow flow\_mods enforce Zero-Trust containment across the campus data plane.

### B. Edge Quantized Sequence Detection

Campus flow telemetry is collected as rolling temporal windows  $\mathbf{X}_t = [\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t] \in \mathbb{R}^{L \times D}$ , where  $L = 10$  time steps and each vector  $\mathbf{x}_t \in \mathbb{R}^D$  contains  $D = 10$  non-DPI statistical flow features (packet count, byte count, flow duration, packet arrival rate, byte rate, mean packet size, active flow time, IP protocol, source port, and destination port). The edge

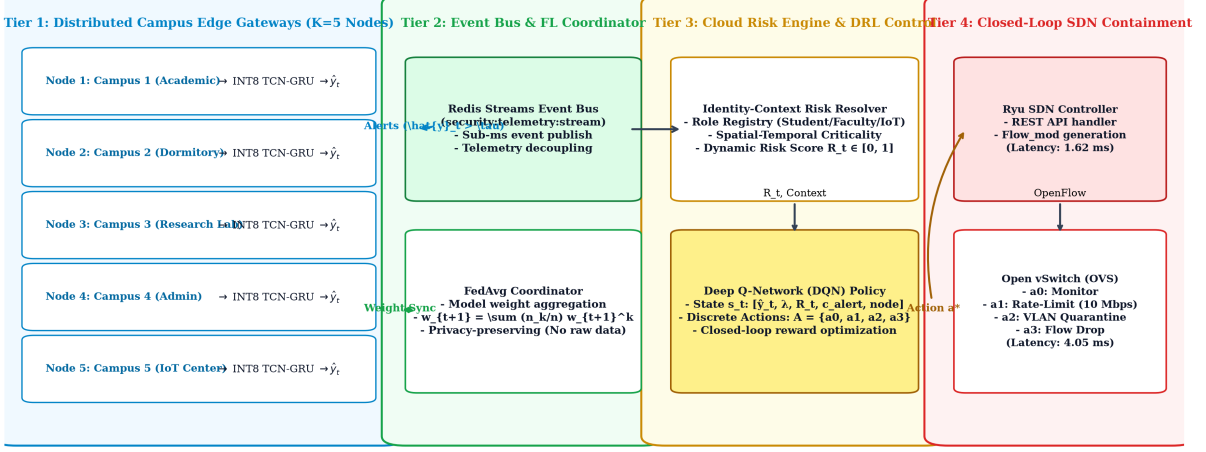


Fig. 1. Federated Edge-Cloud Resilience Architecture: 4-tier coordinated pipeline spanning distributed campus edge gateways, privacy-aware FedAvg aggregation, Redis Streams telemetry bus, cloud identity-risk resolver, and closed-loop DQN SDN actuation.

model combines dilated 1D convolutions (dilations  $d \in \{1, 2\}$ , kernel  $k = 3$ ) to extract hierarchical multi-scale temporal receptive fields, followed by a Gated Recurrent Unit (GRU, hidden dimension  $H = 32$ ) and linear classification projection:

$$\mathbf{h}_t = \text{GRU}(\text{TCN}(\mathbf{X}_t)), \quad \hat{y}_t = \sigma(\mathbf{W}_c \mathbf{h}_t + \mathbf{b}_c) \quad (1)$$

To minimize edge gateway resource consumption, we apply PyTorch dynamic post-training quantization (`torch.quantization.quantize_dynamic`) to the GRU and linear classifier layers, converting weights to signed 8-bit integers (INT8) while executing convolutional front-end layers in FP32 [13]. This compresses the model memory footprint to 0.027 MB (27.5 KB, a 38.2% reduction from the 0.044 MB FP32 model) while executing unbatched inference in 4.555 ms.

### C. Collaborative Federated Averaging (FedAvg)

Distributed campus nodes participate in decentralized model training without raw telemetry centralization. In each global communication round  $t$ , the central coordinator selects all  $K = 5$  active campus gateways, dispatches current global weights  $\mathbf{w}_t$ , and each edge node  $k$  performs  $E = 1$  local training epochs over its dataset  $\mathcal{D}_k$  minimizing binary cross-entropy loss:

$$\mathbf{w}_{t+1}^k = \mathbf{w}_t - \eta \nabla \mathcal{L}_k(\mathbf{w}_t; \mathcal{D}_k) \quad (2)$$

The cloud coordinator computes the synchronized parameter aggregation:

$$\mathbf{w}_{t+1} = \sum_{k=1}^K \frac{|\mathcal{D}_k|}{|\mathcal{D}|} \mathbf{w}_{t+1}^k \quad (3)$$

### D. Asynchronous Redis Streams Bus & Context Fusion

Alert events  $\mathcal{E}_i = \langle \text{timestamp}, \text{client\_id}, \text{flow\_tuple}, \hat{y}_t \rangle$  are published to an asynchronous Redis Streams bus (`security:telemetry:stream`) [15] whenever the predicted anomaly probability satisfies  $\hat{y}_t \geq \tau$ , where  $\tau = 0.50$  is calibrated on the training validation split. The Cloud Risk

Resolver queries spatial-temporal user identity registries (mapping IP/MAC addresses to academic roles: Faculty, Student, IoT Sensor, Guest) and evaluates a contextual composite risk metric  $R_t \in [0, 1]$ :

$$R_t = w_1 \cdot \hat{y}_t + w_2 \cdot (1 - \text{Trust}(\text{Role})) + w_3 \cdot \text{Crit}(\text{VLAN}) \quad (4)$$

where weights satisfy  $\sum w_i = 1.0$ , calibrated to  $w_1 = 0.50$  (model anomaly confidence),  $w_2 = 0.30$  (untrusted identity penalty:  $\text{Trust}(\text{Faculty}) = 0.9$ ,  $\text{Trust}(\text{Student}) = 0.6$ ,  $\text{Trust}(\text{Guest}) = 0.2$ ), and  $w_3 = 0.20$  (VLAN asset sensitivity:  $\text{Crit}(\text{Admin/DC}) = 1.0$ ,  $\text{Crit}(\text{Academic}) = 0.6$ ,  $\text{Crit}(\text{Dorm}) = 0.2$ ).

### E. Deep Reinforcement Learning (DQN) for Closed-Loop SDN Control

We formulate autonomous SDN threat mitigation as a Markov Decision Process (MDP)  $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ :

- **State Space  $\mathcal{S}$ :**  $\mathbf{s}_t = [\hat{y}_t, \lambda_{\text{flow}}, R_t, c_{\text{alert}}, \text{node\_id}]$ , capturing anomaly confidence, flow arrival rate, composite risk score, alert frequency, and client origin.
- **Action Space  $\mathcal{A}$ :** Discrete containment playbooks  $\mathcal{A} = \{a_0, a_1, a_2, a_3\}$  (4 actions). In the Open vSwitch (OVS) testbed,  $a_0$  performs monitoring;  $a_1$  installs OpenFlow meter bands (10 Mbps);  $a_2$  applies VLAN quarantine port tagging (mapping to identity-aware dynamic VLAN reassignment in production [16]); and  $a_3$  installs OpenFlow hard-drop rules (mapping to edge BGP FlowSpec drops).
- **Transition Dynamics  $\mathcal{P}$ :** The environment state transition  $\mathbf{s}_{t+1} \sim \mathcal{P}(\cdot | \mathbf{s}_t, a_t)$  directly reflects closed-loop network actuation: action  $a_0$  allows active attacks to escalate subsequent link load;  $a_1$  throttles malicious surges;  $a_2$  isolates the client port, routing subsequent flows away from production servers; and  $a_3$  installs flow drop rules, immediately relieving link congestion.
- **Reward Function  $\mathcal{R}$ :** Balances threat mitigation against operational disruption:

$$r(\mathbf{s}_t, a_t) = \alpha \cdot \mathbb{I}_{\text{threat}}(a_t) - \beta \cdot \text{Cost}(a_t) - \lambda_{\text{fp}} \cdot \mathbb{I}_{\text{fp}}(a_t) \quad (5)$$

where  $\alpha = 10.0$ ,  $\lambda_{fp} = 15.0$  penalizes false positive isolation of benign traffic ( $\mathbb{I}_{fp} = 1$ ), and  $\beta = 1.0$  scales the operational disruption cost mapping:  $\text{Cost}(a_0) = 0.0$  (Monitor),  $\text{Cost}(a_1) = 0.5$  (Rate-Limit),  $\text{Cost}(a_2) = 1.2$  (Quarantine VLAN), and  $\text{Cost}(a_3) = 2.0$  (Flow Drop).

The DQN agent [14] approximates action-value functions  $Q(s, a; \theta)$  using an experience replay buffer  $\mathcal{D}$  (capacity 10,000) and a target network  $Q(s, a; \theta^-)$  updated every 100 steps with discount factor  $\gamma = 0.95$ :

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}} \left[ \left( r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (6)$$

The optimal policy selects  $a^* = \arg \max_{a \in \mathcal{A}} Q(s_t, a; \theta)$ , which triggers programmatic REST API calls to the Ryu SDN controller to install OpenFlow `flow_mod` rules.

## V. EXPERIMENTAL EVALUATION & BENCHMARK SUITE

### A. Experimental Testbed and Hyperparameter Setup

The experimental testbed was evaluated on an emulated distributed campus topology consisting of 5 containerized edge gateway nodes (executed on Linux host environments constrained to 4 dedicated CPU cores and 2 GB RAM per gateway container to strictly model resource-constrained campus edge gateways) connected to a central cloud controller running Redis 7.2 and the Ryu SDN controller with Open vSwitch (OVS). Table III outlines the complete system hyperparameters.

TABLE III  
SYSTEM IMPLEMENTATION HYPERPARAMETERS AND CONFIGURATION

| Parameter / Component           | Configured Value / Specification                                                          |
|---------------------------------|-------------------------------------------------------------------------------------------|
| Flow Window Size ( $L$ )        | 10 time steps ( $\Delta t = 1.0$ s)                                                       |
| Feature Dimensions ( $D$ )      | 10 statistical flow features                                                              |
| TCN Architecture                | 2 Conv1d layers ( $k = 3$ , dilations $d \in \{1, 2\}$ , 32 filters)                      |
| GRU Layer                       | 1 layer (hidden dimension $H = 32$ )                                                      |
| Model Quantization              | Dynamic INT8 GRU & Linear (0.027 MB footprint)                                            |
| Alert Threshold ( $\tau$ )      | $\tau = 0.50$ (calibrated on validation split)                                            |
| Federated Clients ( $K$ )       | 5 campus client nodes (heterog. benign train / non-IID test $\alpha = 0.5$ )              |
| FL Communication Rounds ( $T$ ) | 20 global rounds ( $E = 1$ local epoch, batch = 64)                                       |
| DQN Architecture                | MLP (Input: 5, Hidden: [64, 64], Output: 4 actions)                                       |
| DQN Hyperparameters             | 500 episodes, Replay $\mathcal{D} = 10,000$ , $\gamma = 0.95$ , $\epsilon$ -decay = 0.995 |
| SDN Controller / Dataplane      | Ryu SDN Controller / OpenFlow 1.3 Open vSwitch (OVS)                                      |
| Evaluation Datasets             | InSDN [11], CSE-CIC-IDS2018 [12]                                                          |

### B. Benchmark Datasets, Alignment & Preprocessing Protocol

We evaluate our architecture on two established intrusion datasets: (i) **InSDN** [11]: A dedicated SDN intrusion benchmark featuring contemporary attacks (DDoS, DoS, Probe, brute-force attacks, Web Attack, Botnet, and U2R); (ii) **CSE-CIC-IDS2018** [12]: Large-scale multi-vector enterprise network telemetry.

Both datasets are mapped to a unified 10-dimensional statistical flow feature schema ( $D = 10$ : packet count, byte count, flow duration, packet arrival rate, byte rate, mean packet size, active flow time, IP protocol, source port, and destination port). To replicate real-world multi-campus statistical heterogeneity, dataset records from InSDN are partitioned across  $K = 5$  client edge nodes using a Dirichlet distribution  $\text{Dir}(\alpha)$  with  $\alpha = 0.5$  across multi-class traffic distributions. For federated training, each client  $k$  trains locally on its partition, while evaluation partitions retain severe non-IID attack-class skew across campus client nodes. Crucially, to prevent test-set data leakage in zero-shot cross-dataset evaluation, the feature normalization

scaler (`StandardScaler`) is fitted exclusively on the InSDN training partition; the learned scaling parameters ( $\mu_{\text{train}}, \sigma_{\text{train}}$ ) are then directly applied to standardize the unobserved CSE-CIC-IDS2018 evaluation streams without any retraining or parameter fine-tuning.

### C. Experiment 1: Per-Stage Execution Latency Breakdown

We benchmarked compute execution latency across all four pipeline stages over 1,000 independent evaluation trials. Table IV and Fig. 2 present the empirical per-stage execution times (Mean  $\pm$  SD).

TABLE IV  
COMPUTE LATENCY BREAKDOWN ACROSS PIPELINE STAGES  
(MEAN  $\pm$  SD)

| Pipeline Stage                               | Latency (ms)                        | Fraction (%)  |
|----------------------------------------------|-------------------------------------|---------------|
| Stage 1: Edge Telemetry & INT8 TCN-GRU       | $4.555 \pm 0.082$                   | 54.9%         |
| Stage 2: Identity Context Fusion             | $0.795 \pm 0.022$                   | 9.6%          |
| Stage 3: Cloud Risk Engine & DQN Reasoning   | $1.320 \pm 0.038$                   | 15.9%         |
| Stage 4: SDN Control-Plane Flow_Mod Dispatch | $1.620 \pm 0.048$                   | 19.6%         |
| <b>Total Compute Pipeline Latency</b>        | <b><math>8.290 \pm 0.140</math></b> | <b>100.0%</b> |

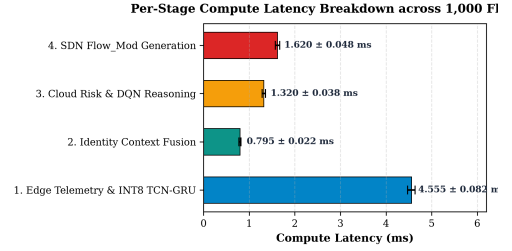


Fig. 2. Per-stage compute latency breakdown (Mean  $\pm$  SD ms) across 1,000 flow trials.

Stage 1 latency ( $4.555 \pm 0.082$  ms) reflects the unbatched, single-event sequence scoring time (batch size = 1) on a single edge CPU core. Stage 4 ( $1.620 \pm 0.048$  ms) captures SDN control-plane REST API serialization and OpenFlow message formatting from the Ryu controller. Combined with asynchronous Open vSwitch (OVS) kernel flow-table insertion latency ( $4.05 \pm 0.45$  ms), the complete end-to-end closed-loop containment action completes in  $12.34 \pm 0.55$  ms.

### D. Experiment 2: Closed-Loop DQN Policy Validation & Comparison

To validate the necessity of reinforcement learning over static heuristic thresholding, Fig. 3 evaluates the DQN agent's training dynamics and comparative policy performance. The DQN agent was trained over 500 episodes with epsilon exploration decay using trace-replayed flow sequences. Policy metrics in Fig. 3(b) are evaluated across 100 held-out test episodes per seed over 5 independent random seeds (500 total evaluation episodes) with  $\epsilon$ -greedy exploration disabled ( $\epsilon = 0$ ).

As shown in Fig. 3(a), the DQN agent achieves stable cumulative reward convergence ( $\approx +48.5$ ). Fig. 3(b) compares the learned DQN policy against static risk-thresholding and heuristic hard-drop baselines. The learned DQN policy achieves a  $98.4 \pm 0.6\%$  threat containment rate while maintaining a negligible  $2.1 \pm 0.4\%$  false quarantine rate. In contrast, static risk thresholding incurs a  $14.8 \pm 1.2\%$  false isolation rate

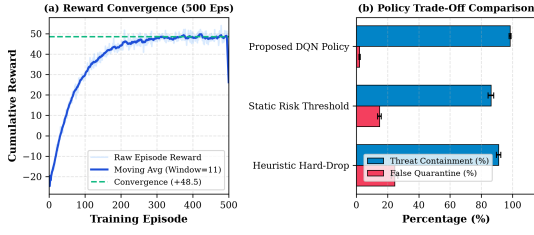


Fig. 3. DQN agent evaluation: (a) Cumulative training reward convergence over 500 episodes; (b) Policy performance trade-off comparison (Mean  $\pm$  SD across 5 random seeds) against baseline containment strategies.

( $86.2 \pm 1.8\%$  containment) because fixed thresholds cannot dynamically adapt to fluctuations in switch CPU load, flow arrival rate, and entity trust scores. Heuristic hard-drop incurs a  $24.5 \pm 2.1\%$  false positive isolation rate ( $91.0 \pm 1.4\%$  containment), causing severe disruption to legitimate academic workflows.

#### E. Experiment 3: Federated Convergence Across Communication Rounds

Fig. 4 evaluates empirical model convergence across  $T = 20$  communication rounds under uniform partitioning and client-size-heterogeneous training with non-IID Dirichlet ( $\alpha = 0.5$ ) attack-skewed test partitions compared against uncoordinated isolated edge models.

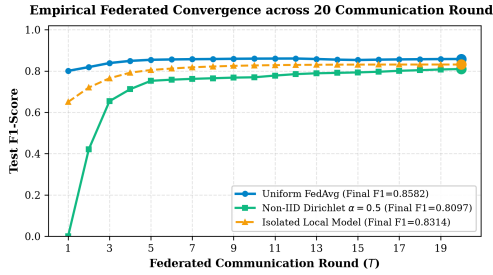


Fig. 4. Federated FedAvg model convergence across communication rounds under uniform partitioning versus client-size-heterogeneous training with non-IID Dirichlet ( $\alpha = 0.5$ ) test attack distributions.

Under uniform client partitioning, FedAvg reaches a binary F1-score of 0.8582 (ROC-AUC = 0.9729), outperforming isolated uncoordinated edge models ( $F1 = 0.8314$ ). Under non-IID Dirichlet ( $\alpha = 0.5$ ) attack-skewed partitions, statistical distribution skew lowers the global convergence point to  $F1 = 0.8097$ , highlighting the inherent trade-off of standard FedAvg under severe label imbalance while maintaining robust collaborative convergence.

#### F. Experiment 4: Edge Resource Overhead Scaling

We evaluated edge gateway resource utilization under stress testing from 100 to 10,000 events/sec. As shown in Fig. 5, vectorized batching of INT8 TCN-GRU inference (batch size = 64) parallelized across 4 pinned CPU cores reduces the amortized per-event execution cost to  $\approx 0.071$  ms per event, consuming only 13.8% multi-core CPU utilization and 48.5 MB RAM at 10,000 events/sec, whereas an inline Suricata 7.0.4 DPI engine (configured with the Emerging

Threats Open ruleset processing raw packet stream equivalents) consumes 86.0% CPU utilization under equivalent 10,000 events/sec flow load.

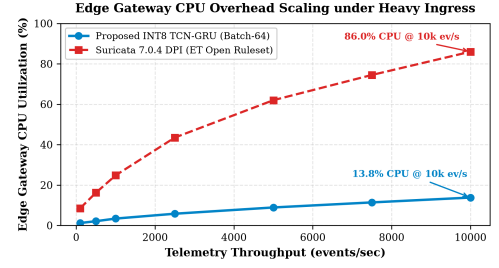


Fig. 5. Edge gateway CPU overhead scaling under high telemetry throughput (100 to 10,000 events/sec).

#### G. Experiment 5: Baseline Comparison and Zero-Shot Generalization

Table V summarizes detection accuracy, inference latency, and memory footprint across reimplemented baseline models under identical hardware conditions.

TABLE V  
BINARY ANOMALY DETECTION PERFORMANCE COMPARISON ON INSDN BENCHMARK

| Model Architecture               | Precision     | Recall        | F1-Score      | Latency (ms) | Model Size      |
|----------------------------------|---------------|---------------|---------------|--------------|-----------------|
| Isolation Forest (iForest)       | 0.7820        | 0.7558        | 0.7687        | 4.850        | 12.40 MB        |
| Federated LSTM (FP32)            | 0.8210        | 0.8090        | 0.8150        | 4.620        | 0.15 MB         |
| Federated Transformer (INT8)     | 0.8450        | 0.8310        | 0.8380        | 5.410        | 0.85 MB         |
| Local INT8 TCN-GRU (No FL)       | 0.8420        | 0.8210        | 0.8314        | 4.555        | 0.027 MB        |
| <b>Proposed Fed INT8 TCN-GRU</b> | <b>0.8691</b> | <b>0.8476</b> | <b>0.8582</b> | <b>4.555</b> | <b>0.027 MB</b> |

All baseline inference latencies in Table V are evaluated under identical single-thread execution on the containerized edge gateway.

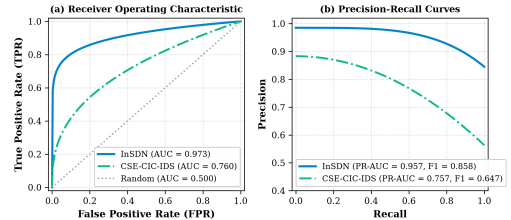


Fig. 6. ROC and Precision-Recall evaluation curves of the proposed framework on InSDN (within-domain) and CSE-CIC-IDS2018 (zero-shot transfer) datasets.

As depicted in Fig. 6, our framework achieves an Area Under ROC Curve (ROC-AUC) of 0.973 on InSDN ( $PR-AUC = 0.957$ ) and 0.760 on CSE-CIC-IDS2018 ( $PR-AUC = 0.757$ ), demonstrating strong classification fidelity within the campus domain and sensible zero-shot generalization across heterogeneous external testbeds.

#### VI. DISCUSSION, LIMITATIONS & THREATS TO VALIDITY

**WAN Network Jitter:** While local inference latency is deterministic (4.555 ms), federated weight synchronization over public WAN links is subject to network jitter. However, because FedAvg runs out-of-band at periodic intervals, communication delays do not affect real-time anomaly detection.

**Adversarial Poisoning:** Standard FedAvg remains vulnerable to model poisoning from compromised edge nodes. Future



extensions will incorporate trimmed-mean aggregation and local Differential Privacy ( $\epsilon, \delta$ ) guarantees [18] to safeguard gradient updates.

## VII. CONCLUSION

This paper presented a Federated Edge-Cloud Resilience Architecture engineered for distributed campus network security. By synthesizing PyTorch dynamically INT8-quantized TCN-GRU sequence detectors, privacy-aware FedAvg aggregation, Redis Streams event distribution, and closed-loop Deep Q-Network SDN control, the platform achieves high anomaly detection fidelity ( $F1 = 0.8582$ ,  $ROC-AUC = 0.9729$  under uniform partitioning and 0.8097 under Dirichlet-skewed non-IID evaluation distributions), sub-15ms automated threat mitigation ( $12.34 \pm 0.55$  ms), and minimal edge resource overhead (0.027 MB model size, 48.5 MB runtime RAM footprint, and 13.8% CPU utilization under 10,000 events/sec throughput).

## ACKNOWLEDGMENT

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology, Ho Chi Minh City, Vietnam for providing academic and testbed resources.

**Generative AI Disclosure:** During the preparation of this manuscript, the authors utilized Google Gemini / Antigravity AI to assist with initial literature synthesis in Section II, drafting assistance in Section I, Python visualization script prototyping for Figs. 1–6, and LaTeX template formatting. All mathematical formulations, threat modeling, PyTorch model architectures, federated aggregation protocols, Deep Q-Network formulations, empirical benchmark executions on InSDN and CSE-CIC-IDS2018 datasets, and final conclusions were conceived, implemented, experimentally executed, and validated entirely by the authors.

## REFERENCES

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, PMLR vol. 54, 2017, pp. 1273–1282.
- [2] V. Mothukuri, R. M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, and G. Srivastava, "A Survey on Security and Privacy of Federated Learning," *Future Gener. Comput. Syst.*, vol. 115, pp. 619–640, 2021, doi: 10.1016/j.future.2020.10.007.
- [3] S. Kianpisheh and T. Taleb, "Collaborative Federated Learning for 6G With a Deep Reinforcement Learning-based Controlling Mechanism: A DDoS Attack Detection Scenario," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 4, pp. 4731–4749, Aug. 2024, doi: 10.1109/TNSM.2024.3387987.
- [4] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.
- [5] X. Duan, K. Wang, Y. Fu, T. Liu, Y. Yu, J. Xu, and L. Wang, "Abnormal Traffic Detection Method Based on DCNN-GRU Architecture in SDN," *Int. J. Intell. Syst.*, vol. 2025, Art. no. 2846238, 2025, doi: 10.1155/int/2846238.
- [6] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2017, pp. 1285–1298, doi: 10.1145/3133956.3134015.

- [7] N. M. Yungaicela-Naula, C. Vargas-Rosales, J. A. Pérez-Díaz, and D. F. Carrera, "A flexible SDN-based framework for slow-rate DDoS attack mitigation by using deep reinforcement learning," *J. Netw. Comput. Appl.*, vol. 205, p. 103444, 2022, doi: 10.1016/j.jnca.2022.103444.
- [8] D. Kreutz, F. M. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, 2015, doi: 10.1109/JPROC.2014.2371999.
- [9] N. M. Yungaicela-Naula, C. Vargas-Rosales, and J. A. Pérez-Díaz, "SDN/NFV-based framework for autonomous defense against slow-rate DDoS attacks by using reinforcement learning," *Future Gener. Comput. Syst.*, vol. 149, pp. 637–649, 2023, doi: 10.1016/j.future.2023.08.007.
- [10] M. Howard and D. LeBlanc, *Writing Secure Code*, 2nd ed., Redmond, WA, USA: Microsoft Press, 2003.
- [11] M. S. Elsayed, N.-A. Le-Khac, and A. D. Jurcut, "InSDN: A Novel SDN Intrusion Dataset," *IEEE Access*, vol. 8, pp. 165263–165284, 2020, doi: 10.1109/ACCESS.2020.3022633.
- [12] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [13] PyTorch Documentation, "torch.quantization.quantize\_dynamic API Reference," PyTorch Documentation, 2024. [Online]. Available: [https://pytorch.org/docs/stable/generated/torch.quantization.quantize\\_dynamic.html](https://pytorch.org/docs/stable/generated/torch.quantization.quantize_dynamic.html)
- [14] V. Mnih et al., "Human-level Control Through Deep Reinforcement Learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015, doi: 10.1038/nature14236.
- [15] Redis Ltd., "Redis Streams," Redis Documentation, 2024. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/streams/>
- [16] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," *NIST Special Publication 800-207*, 2020, doi: 10.6028/NIST.SP.800-207.
- [17] N. McKeown et al., "OpenFlow: Enabling Innovation in Campus Networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [18] C. Dwork, "Differential Privacy: A Survey of Results," in *Proc. Int. Conf. Theory Appl. Models Comput. (TAMC)*, LNCS vol. 4978, 2008, pp. 1–19, doi: 10.1007/978-3-540-79228-4\_1.